MongoDB (Configuration, Local Setup, and Usage)

This project uses **MongoDB** as the primary datastore for the Django backend.

- Django connects via **`django-mongodb-backend`** (so Django models/migrations work against MongoDB).
- A small amount of code also uses **PyMongo** directly for specific collections.

Where MongoDB is configured

Django settings

MongoDB settings live in `backend/config/settings.py`.

Key behavior:

- If `MONGODB_HOST` is set, Django uses `MONGODB_HOST` + `MONGODB_PORT`.
- Else if `MONGODB_URI` is set, Django parses the host/port from it.
- Else it defaults to `127.0.0.1:27017`.

The configured database is:

- `DATABASES["default"]["NAME"] = "legal_chatbot_db"`

The active connection URI used by PyMongo helpers is normalized to match host/port:

- `MONGODB_URI = mongodb://<host>:<port>`

App config overrides (Django + Mongo quirks)

`backend/apps/mongo_db_config.py` provides:

- `MongoAdminConfig`, `MongoAuthConfig`, `MongoContentTypesConfig`
- A `ready()` hook that **disconnects** Django's default `create_permissions` post-migrate signal for `AuthConfig` (it can crash with MongoDB content-types/permissions semantics).

These are registered in `INSTALLED_APPS` (see `backend/config/settings.py`).

Environment variables

Backend `.env`

`backend/.env.example` shows the expected Mongo variables:

- `MONGODB_HOST` (default local dev: `127.0.0.1`)
- `MONGODB_PORT` (default: `27017`)
- Optional `MONGODB_URI` (if set, overrides host/port parsing rules)
- `MONGODB_DB_NAME` (used by PyMongo helper functions; **separate** from Django's `legal_chatbot_db`)

Important note about names:

- Django models use the Mongo database named **`legal_chatbot_db`**.
- `MONGODB_DB_NAME` is used by PyMongo helpers in `backend/apps/mongo_db_config.py` to pick a database for those collections.
- For simplicity, you usually want these to align (e.g., set `MONGODB_DB_NAME=legal_chatbot_db`) unless you intentionally split data across databases.

Docker Compose (recommended for local dev)

The root `docker-compose.yml` defines a `mongodb` service:

- Image: `mongo:7`
- Port mapping: `27017:27017`
- Volume: `mongo_data:/data/db`
- Healthcheck: `mongosh --eval "db.adminCommand('ping')"`

The `backend` service sets:

- `MONGODB_HOST=mongodb`
- `MONGODB_PORT=27017`

This is critical: inside Compose, containers reach MongoDB via the **service name** `mongodb`, not `127.0.0.1`.

Local setup steps

Option A: Run MongoDB with Docker (host-based Django)

1. Start MongoDB:

```
docker compose up -d mongodb
```

2. In `backend/.env`, use:

```
MONGODB_HOST=127.0.0.1
MONGODB_PORT=27017
```

3. Run migrations:

```
cd backend
python manage.py migrate
```

4. Start Django:

```
python manage.py runserver 8000
```

Option B: Run the full stack with Docker Compose

If you run Django in Docker Compose, you do not need `127.0.0.1` for Mongo; Compose injects:

- `MONGODB_HOST=mongodb`

Bring everything up (profile `app` is used in this repo):

```
docker compose --profile app up --build
```

How data is stored in MongoDB

Django models (primary application data)

The main application entities (users, chats, messages, topic mastery, documents metadata, etc.) are modeled as Django models under:

- `backend/apps/users/models.py`
- `backend/apps/chat/models.py`
- `backend/apps/documents/models.py` (if present) / migrations
- `backend/apps/courses/models.py`

These are stored via `django-mongodb-backend` in the database named `legal_chatbot_db`.

PyMongo collections (direct access)

`backend/apps/mongo_db_config.py` exposes helper functions:

- `get_mongo_db()`
- `get_collection(collection_name)`
- `get_knowledge_base_collection()` (with a compound unique index on `legal_domain` + `jurisdiction`)
- `get_self_assessments_collection()` (unique index on `chat_id`)

This is used when the code wants:

- explicit index management
- simple collection operations without Django models

Troubleshooting

“Cannot connect to MongoDB” (common causes)

- **Django on host, Mongo in Docker**: use `MONGODB_HOST=127.0.0.1`, not `mongodb`.
- **Django in Docker, Mongo in Docker**: use `MONGODB_HOST=mongodb`, not `127.0.0.1`.
- Port mismatch: ensure Mongo is listening on `27017` and port mapping exists.

Migrations behave differently than SQL

MongoDB-backed Django works, but some Django internals (notably permissions/content-types) can behave differently. This repo mitigates one known issue by disconnecting the permissions creation signal (see `MongoAuthConfig.ready()`).